



PROJET DE FIN D'ÉTUDES

pour obtenir le diplôme de

l' UNIVERSITÉ GALATASARAY

Spécialité : **Mathématiques**

Approximation de Matrices de Rang Faible pour la Compression d'Images avec la DVS

Préparé par : **BERRİN KARAKAYA**

Réponsable : **OĞUZHAN KAYA**

Table des Matières

Objectifs du Projet	1
Introduction	1
Théorème Spectral	2
Décomposition En Valeurs Singulières	8
Géométrie de la DVS	10
Le Quotient de Rayleigh	11
Approximation Matricielle	14
Application de la DVS à la compression d'images	16
Conclusion	21
Annexes	22

Objectifs du Projet

Dans ce projet, nous étudions comment les outils de la Théorème Spectral peuvent être utilisés pour analyser des matrices de grande dimension. L'objectif est de comprendre comment obtenir une approximation de rang faible d'une image numérique (représentée par une matrice), tout en minimisant la perte d'information par rapport à la représentation initiale.

Dans un premier temps, nous rappelons les notions de base de la Théorème Spectral. Nous montrons ensuite comment cette approche conduit à la décomposition en valeurs singulières dans le cas des matrices rectangulaires.

Enfin, nous appliquons ces outils à des images numériques afin de construire des approximations de ces images en utilisant une quantité réduite de données. Ce travail permet d'illustrer, d'un point de vue mathématique, comment la Décomposition en Valeurs Singulières conduit à une réduction de dimension.

Introduction

Comprendre une application linéaire peut être difficile, car la matrice qui la représente peut contenir plusieurs effets géométriques dans des espaces de grande dimension. Le Théorème Spectral nous aide à comprendre ces transformations d'une manière simple. Il montre qu'une matrice symétrique peut être écrite avec des vecteurs propres et des valeurs propres. Les valeurs propres indiquent combien la transformation agrandit ou réduit dans une direction. Les vecteurs propres sont des vecteurs dont la direction ne change pas sous cette transformation. Ainsi, la transformation devient plus claire. Les matrices orthogonales sont utiles car elles gardent les distances et les angles. Elles représentent des rotations ou des symétries. Elles ne déforment pas l'espace et sont très stables dans les calculs, ce qui est important en informatique. La matrice diagonale obtenue avec les valeurs propres permet de simplifier les calculs. Les calculs de puissance ou l'inverse d'une telle matrice sont plus faciles.

Cependant, dans de nombreuses applications, les objets étudiés ne sont pas représentés par des matrices carrées ou symétriques. Pour traiter ce type de matrices, on peut construire des matrices symétriques associées, comme $A^T A$, auxquelles le Théorème Spectral peut être appliqué. Cette approche conduit naturellement à la Décomposition en Valeurs Singulières, qui étend la notion de diagonalisation au cas des matrices rectangulaires. La décomposition en valeurs singulières permet d'écrire une matrice comme une somme de matrices de rang 1. La Décomposition en Valeurs Singulières permet de représenter la matrice associée à l'image originale à l'aide d'une quantité réduite d'informations, tout en restant proche de la matrice initiale. Cette démarche s'inscrit dans le cadre de l'approximation de matrices et conduit à une réduction de dimension, puisque l'image est décrite avec moins d'informations que dans sa représentation initiale.

Dans cette thèse, nous étudierons la Théorème Spectral, la Décomposition en Valeurs Singulières et l'approximation matricielle. En appliquant ces outils aux images numériques, nous démontrerons comment on peut approximer à une image peut être représentée par une matrice par des matrices d'ordre inférieur. L'objectif de ce travail est de fournir une compréhension mathématique claire de la manière dont la réduction de dimensionnalité peut être obtenue à partir de la structure des matrices associées aux images.

Théorème Spectral

Le Théorème Spectral permet de représenter l'application linéaire $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ associée à $A \in \mathbb{R}^{n \times n}$ une matrice symétrique réelle si et seulement si A peut être exprimée sous la forme :

$$A = QDQ^\top$$

où Q est une matrice orthogonale dont les colonnes $\{u_1, \dots, u_n\}$ sont les vecteurs propres de A . Ces colonnes forment une base orthonormale pour $\mathbb{R}^n$. Puisque Q est orthogonale, son inverse est égal à sa transposée, $Q^{-1} = Q^\top$.

D est une matrice diagonale dont les éléments diagonaux sont les valeurs propres de A , correspondant aux vecteurs propres dans Q dans le même ordre.

Autrement dit, on peut écrire la matrice A simplement comme une somme de ses valeurs propres et vecteurs propres :

$$A = \lambda_1 u_1 u_1^\top + \lambda_2 u_2 u_2^\top + \dots + \lambda_n u_n u_n^\top = \sum_{i=1}^n \lambda_i u_i u_i^\top.$$

Exemple 1. Considérons la matrice symétrique $A \in \mathbb{R}^{3 \times 3}$ suivante :

$$A = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix}.$$

Tout d'abord on détermine $\text{spec}(A)$. Le polynôme caractéristique est

$$\det(A - \lambda I) = \det \begin{pmatrix} 2 - \lambda & 0 & 0 \\ 0 & 3 - \lambda & 1 \\ 0 & 1 & 3 - \lambda \end{pmatrix} = 0.$$

On obtient :

$$(2 - \lambda)((3 - \lambda)^2 - 1) = 0 \implies (2 - \lambda)(2 - \lambda)(4 - \lambda) = 0 \implies (2 - \lambda)^2(4 - \lambda) = 0.$$

Ainsi, l'ensemble des valeurs propres est $\text{spec}(A) = \{2, 4\}$. Ensuite, on trouve les vecteurs v non-nuls qui satisfont $(A - \lambda I)v = 0$.

Pour $\lambda = 2$:

$$(A - 2I)v = 0 \implies \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \implies y + z = 0.$$

Les vecteurs de base sont : $v_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$, $v_2 = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}$.

Pour $\lambda = 4$:

$$(A - 4I)v = 0 \implies \begin{pmatrix} -2 & 0 & 0 \\ 0 & -1 & 1 \\ 0 & 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \implies y = z.$$

Le vecteur de base est : $v_3 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$.

On va écrire une matrice orthogonale Q . Cela signifie que les vecteurs propres que nous plaçons dans Q peuvent être orthogonaux entre eux. Ainsi, si un vecteur propre n'est pas de longueur 1, nous le normalisons en le divisant par sa norme $\|v\|$: Nous normalisons les vecteurs v en $u = v/\|v\|$:

$$u_1 = \frac{v_1}{\|v_1\|} = \frac{1}{1} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$u_2 = \frac{v_2}{\|v_2\|} = \frac{1}{\sqrt{0^2 + 1^2 + (-1)^2}} \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$$

$$u_3 = \frac{v_3}{\|v_3\|} = \frac{1}{\sqrt{0^2 + 1^2 + 1^2}} \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$$

Q est la matrice orthogonale formée des vecteurs propres normalisés $\{u_1, u_2, u_3\}$:

$$Q = [u_1 \ u_2 \ u_3] = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1/\sqrt{2} & 1/\sqrt{2} \\ 0 & -1/\sqrt{2} & 1/\sqrt{2} \end{pmatrix}.$$

D est la matrice diagonale des valeurs propres correspondantes :

$$D = \text{diag}(2, 2, 4) = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 4 \end{pmatrix}.$$

Le Théorème Spectral est vérifié :

$$\boxed{A = QDQ^\top}$$

Lemme 1 (Valeurs propres réelles). *Toutes les valeurs propres d'une matrice symétrique réelle sont réelles.*

Démonstration. Soit $\lambda \in \mathbb{C}$ une valeur propre de la matrice réelle symétrique A et $x \in \mathbb{C}^n$ un vecteur propre associé non nul ($x \neq 0$), tel que $Ax = \lambda x$.

On considère $\mathbb{C}^n$ muni du produit scalaire hermitien usuel noté $\langle \cdot, \cdot \rangle$. Puisque A est réelle et symétrique ($A = A^\top$), on a $A = \overline{A}^\top = A^*$.

Par la propriété de l'adjoint dans un produit scalaire ($\langle Ax, y \rangle = \langle x, A^*y \rangle$), nous obtenons l'égalité suivante :

$$\langle Ax, x \rangle = \langle x, A^*x \rangle = \langle x, Ax \rangle.$$

En remplaçant $Ax = \lambda x$, on obtient :

$$\langle \lambda x, x \rangle = \langle x, \lambda x \rangle$$

En utilisant les propriétés de linéarité du produit scalaire :

$$\lambda \langle x, x \rangle = \bar{\lambda} \langle x, x \rangle$$

Comme $x \neq 0$, on a $\langle x, x \rangle = \|x\|^2 > 0$. Nous pouvons donc simplifier l'équation :

$$\lambda = \bar{\lambda}$$

Ceci implique que la partie imaginaire de λ est nulle, donc $\lambda \in \mathbb{R}$. □

Lemme 2 (Vecteurs propres orthogonaux). *Les vecteurs propres correspondant à des valeurs propres distinctes d'une matrice symétrique sont orthogonaux.*

Démonstration. Soient λ_1 et λ_2 deux valeurs propres distinctes correspondant respectivement aux vecteurs propres x_1 et x_2 , tels que $Ax_1 = \lambda_1 x_1$ et $Ax_2 = \lambda_2 x_2$.

Nous allons établir deux égalités pour comparer les produits scalaires. Prenons la première équation $Ax_1 = \lambda_1 x_1$. Appliquons d'abord la transposée des deux côtés, puis multiplions à droite par le vecteur x_2 :

$$(Ax_1)^\top = (\lambda_1 x_1)^\top \implies x_1^\top A^\top = \lambda_1 x_1^\top$$

Puisque A est symétrique ($A = A^\top$), on remplace $A^\top$ par A . En multipliant ensuite à droite par x_2 , on obtient :

$$x_1^\top Ax_2 = \lambda_1 x_1^\top x_2$$

Partons maintenant de la deuxième équation $Ax_2 = \lambda_2 x_2$. Cette fois, multiplions directement à gauche par le vecteur transposé $x_1^\top$:

$$x_1^\top (Ax_2) = x_1^\top (\lambda_2 x_2) \implies x_1^\top Ax_2 = \lambda_2 x_1^\top x_2$$

En égalisant ces deux expressions, nous obtenons :

$$\lambda_1 x_1^\top x_2 = \lambda_2 x_1^\top x_2$$

Ce qui équivaut à :

$$(\lambda_1 - \lambda_2) x_1^\top x_2 = 0$$

Puisque les valeurs propres sont distinctes ($\lambda_1 \neq \lambda_2$), on a nécessairement $\lambda_1 - \lambda_2 \neq 0$. Par conséquent, le produit scalaire doit être nul :

$$x_1^\top x_2 = 0$$

Cela prouve que les vecteurs propres x_1 et x_2 sont orthogonaux. □

Lemme 3 (Indépendance des Vecteurs Propres). *Si une matrice $A \in \mathbb{R}^{n \times n}$ possède k valeurs propres distinctes, alors tout ensemble de k vecteurs propres correspondants (non nuls) est linéairement indépendant.*

Démonstration. Nous prouvons ce théorème par récurrence.

Initialisation : D'abord, montrons que deux vecteurs propres correspondant à des valeurs propres distinctes sont linéairement indépendants. Soient v_1 et v_2 des vecteurs propres associés respectivement aux valeurs propres distinctes λ_1 et λ_2 . Supposons, par l'absurde, que v_1 et v_2 sont linéairement dépendants. Alors il existe deux scalaires α_1, α_2 (non tous les deux nuls) tels que :

$$\alpha_1 v_1 + \alpha_2 v_2 = 0 \tag{1}$$

En multipliant (1) à gauche par A , on obtient :

$$\alpha_1 \lambda_1 v_1 + \alpha_2 \lambda_2 v_2 = 0 \tag{2}$$

De même, en multipliant (1) par λ_2 , on obtient :

$$\alpha_1 \lambda_2 v_1 + \alpha_2 \lambda_2 v_2 = 0 \tag{3}$$

En soustrayant l'équation (2) de l'équation (3), on trouve :

$$\alpha_1 (\lambda_2 - \lambda_1) v_1 = 0.$$

Puisque $\lambda_2 \neq \lambda_1$ et $v_1 \neq 0$, nous devons avoir $\alpha_1 = 0$. Comme $v_2 \neq 0$, en remplaçant $\alpha_1 = 0$ dans (1), on obtient $\alpha_2 = 0$, ce qui mène à une contradiction. Ainsi, v_1 et v_2 sont linéairement indépendants.

Hérédité : Supposons maintenant que tout ensemble de $j < k$ vecteurs propres correspondant à des valeurs propres distinctes est linéairement indépendant. Nous voulons montrer que $j + 1$ vecteurs propres sont aussi linéairement indépendants.

Soient $v_1, v_2, \dots, v_j$ des vecteurs propres linéairement indépendants correspondant à des valeurs propres distinctes $\lambda_1, \lambda_2, \dots, \lambda_j$. Supposons par l'absurde qu'un vecteur propre supplémentaire v_{j+1} , correspondant à une valeur propre différente λ_{j+1} , est linéairement dépendant de $v_1, \dots, v_j$. Alors il existe des scalaires $\alpha_1, \alpha_2, \dots, \alpha_j$ non tous nuls tels que :

$$v_{j+1} = \alpha_1 v_1 + \alpha_2 v_2 + \dots + \alpha_j v_j \quad (4)$$

En multipliant (4) à gauche par A :

$$\lambda_{j+1} v_{j+1} = \alpha_1 \lambda_1 v_1 + \alpha_2 \lambda_2 v_2 + \dots + \alpha_j \lambda_j v_j \quad (5)$$

En multipliant (4) par le scalaire λ_{j+1} :

$$\lambda_{j+1} v_{j+1} = \alpha_1 \lambda_{j+1} v_1 + \alpha_2 \lambda_{j+1} v_2 + \dots + \alpha_j \lambda_{j+1} v_j \quad (6)$$

En soustrayant ces deux équations, on obtient :

$$\alpha_1 (\lambda_{j+1} - \lambda_1) v_1 + \alpha_2 (\lambda_{j+1} - \lambda_2) v_2 + \dots + \alpha_j (\lambda_{j+1} - \lambda_j) v_j = 0.$$

D'après l'hypothèse de récurrence, les vecteurs v_i (pour $i \leq j$) sont linéairement indépendants. Par conséquent, tous les coefficients doivent être nuls :

$$\alpha_i (\lambda_{j+1} - \lambda_i) = 0 \quad \text{pour tout } i \in \{1, \dots, j\}.$$

Comme les valeurs propres sont distinctes ($\lambda_{j+1} \neq \lambda_i$), on a nécessairement $\alpha_i = 0$ pour tout i . Cela contredit l'hypothèse que les vecteurs sont dépendants. Ainsi, les vecteurs propres $v_1, \dots, v_{j+1}$ sont linéairement indépendants.

Par récurrence, tout ensemble de k vecteurs propres correspondant à k valeurs propres distinctes est linéairement indépendant. $\square$

Exemple 2. Considérons la matrice 3×3 définie par :

$$A = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 2 & -2 \\ 0 & -2 & 2 \end{pmatrix}$$

Elle possède trois valeurs propres (calculons $\det(A - \lambda I) = (4 - \lambda)(\lambda^2 - 4\lambda)$) :

$$\lambda_1 = 4, \quad \lambda_2 = 4, \quad \lambda_3 = 0$$

On observe ici une valeur propre répétée $\lambda = 4$ avec une multiplicité algébrique de $m_{\lambda=4} = 2$, et une valeur propre $\lambda = 0$ avec $m_{\lambda=0} = 1$.

Nous allons calculer les espaces propres pour vérifier si la multiplicité géométrique est égale à la multiplicité algébrique.

Pour la valeur propre répétée $\lambda = 4$: Nous cherchons l'espace propre E_4 en résolvant $(A - 4I)v = 0$ pour $v = (x, y, z)^\top$:

$$(A - 4I)v = \begin{pmatrix} 0 & 0 & 0 \\ 0 & -2 & -2 \\ 0 & -2 & -2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Ce système nous donne l'équation $-2y - 2z = 0 \implies y = -z$. La variable x n'apparaît pas dans les contraintes, elle est donc libre, tout comme z . Puisque nous avons deux variables libres (x et z), nous pouvons former deux vecteurs de base linéairement indépendants :

$$v_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad (\text{avec } x = 1, z = 0), \quad v_2 = \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix} \quad (\text{avec } x = 0, z = 1)$$

La dimension de l'espace propre E_4 est donc 2.

$$\text{Multiplicité Géométrique } (g_{\lambda=4}) = \dim(E_4) = 2.$$

On constate que $g_{\lambda=4} = m_{\lambda=4}$.

Pour la valeur propre simple $\lambda = 0$: Nous cherchons l'espace propre E_0 en résolvant $(A - 0I)v = 0$:

$$Av = \begin{pmatrix} 4 & 0 & 0 \\ 0 & 2 & -2 \\ 0 & -2 & 2 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

Ce système impose $4x = 0 \implies x = 0$. La deuxième ligne donne $2y - 2z = 0 \implies y = z$. En posant $z = 1$, on obtient $y = 1$.

$$v_3 = \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$

La dimension de l'espace propre E_0 est 1.

$$\text{Multiplicité Géométrique } (g_{\lambda=0}) = \dim(E_0) = 1.$$

Ici aussi, $g_{\lambda=0} = m_{\lambda=0}$.

Pour chaque valeur propre, la multiplicité géométrique est égale à la multiplicité algébrique ($g_{\lambda} = m_{\lambda}$). Cela signifie que malgré la présence d'une racine répétée, la matrice possède un nombre suffisant de vecteurs propres linéairement indépendants (v_1, v_2, v_3).

Lemme 4 (Rang du produit de matrices). Soient $A \in \mathbb{R}^{m \times n}$ et $B \in \mathbb{R}^{n \times k}$. Le rang de leur produit $AB \in \mathbb{R}^{m \times k}$ satisfait l'inégalité suivante :

$$\text{rang}(AB) \leq \min\{\text{rang}(A), \text{rang}(B)\}.$$

Démonstration. Considérons le produit matriciel AB :

- Chaque ligne de AB est une combinaison linéaire des lignes de B , ce qui implique que l'espace ligne de AB est contenu dans celui de B . Par conséquent, $\text{rang}(AB) \leq \text{rang}(B)$.
- De même, chaque colonne de AB est une combinaison linéaire des colonnes de A , donc l'espace colonne de AB est contenu dans celui de A . D'où $\text{rang}(AB) \leq \text{rang}(A)$.

En combinant ces observations, on conclut que $\text{rang}(AB) \leq \min\{\text{rang}(A), \text{rang}(B)\}$. $\square$

Lemme 5 (Rang des matrices symétriques). Si A est une matrice réelle symétrique $n \times n$, alors $\text{rang}(A)$ est égal au nombre total de valeurs propres non nulles de A . De plus, l'espace colonne $C(A)$ est le sous-espace linéaire engendré par les vecteurs propres de A correspondant à ses valeurs propres non nulles.

Démonstration. Toute matrice symétrique A peut être exprimée sous sa forme spectrale $A = Q\Lambda Q^\top$, où Q est une matrice orthogonale et Λ est une matrice diagonale contenant les valeurs propres de A . En utilisant le lemme précédent, nous procédons comme suit :

- À partir de $A = Q\Lambda Q^\top$, on a $\text{rang}(A) \leq \text{rang}(Q\Lambda) \leq \text{rang}(\Lambda)$.
- À partir de $\Lambda = Q^\top A Q$, on a $\text{rang}(\Lambda) \leq \text{rang}(Q^\top A) \leq \text{rang}(A)$.

Cela implique que $\text{rang}(A) = \text{rang}(\Lambda)$, ce qui est égal au nombre total de valeurs propres non nulles de A . $\square$

Exemple 3. Soit la matrice symétrique réelle :

$$A = \begin{bmatrix} 8 & -2 & 2 \\ -2 & 5 & 4 \\ 2 & 4 & 5 \end{bmatrix}$$

Le polynôme caractéristique $P_A(\lambda) = -\lambda(\lambda - 9)^2$ donne les valeurs propres :

$$\lambda_1 = 0 \quad (m_0 = 1), \quad \lambda_{2,3} = 9 \quad (m_9 = 2).$$

Les espaces propres associés sont :

- Pour $\lambda = 0$: $E_0 = \text{vect}(v_1)$ avec $v_1 = (1, 2, -2)^\top$. ($g_0 = m_0 = 1$).
- Pour $\lambda = 9$: $E_9 = \text{vect}(v_2, v_3)$ avec $v_2 = (-2, 1, 0)^\top$, $v_3 = (2, 0, 1)^\top$. ($g_9 = m_9 = 2$).

La matrice est diagonalisable car les multiplicités géométriques et algébriques sont égales.

Les valeurs propres sur la diagonale de Λ sont $\{0, 9, 9\}$. Puisqu'il y a exactement 2 valeurs non nulles, le rang est :

$$\text{rang}(A) = 2.$$

Comme indiqué dans le lemme, l'espace colonne est engendré par les vecteurs propres correspondant aux valeurs propres non nulles. Ici, le vecteur v_1 correspond à $\lambda = 0$, donc il ne contribue pas à l'espace colonne. L'espace colonne est donc engendré uniquement par v_2 et v_3 :

$$C(A) = \text{vect}(v_2, v_3).$$

Cet exemple montre que, même en présence de valeurs propres répétées, le rang de la matrice symétrique est entièrement déterminé par les sous-espaces propres associés aux valeurs propres non nulles. [3]

Remarque 1 (Unicité de la Théorème Spectrale). *Il est important de noter que la Théorème Spectral d'une matrice n'est généralement pas unique. L'une des raisons est la présence de valeurs propres répétées.*

Lorsque deux ou plusieurs valeurs propres λ_i et λ_j sont identiques, l'échange de leurs vecteurs propres correspondants dans la matrice orthogonale Q résulte en une décomposition différente qui reste mathématiquement valide et équivalente. Cependant, les sous-espaces propres associés à chaque valeur propre restent fixes. Cela signifie que, bien que le choix des vecteurs propres au sein de chaque sous-espace puisse varier, la décomposition en termes de sous-espaces propres est unique.

Dans l'exercice que nous venons de résoudre avec la matrice :

$$A = \begin{bmatrix} 8 & -2 & 2 \\ -2 & 5 & 4 \\ 2 & 4 & 5 \end{bmatrix}$$

nous avons observé une valeur propre répétée $\lambda = 9$ de multiplicité 2.

- **Non-unicité de la base :** Nous avons choisi une base orthonormée $\{u_2, u_3\}$ pour construire la matrice Q . Cependant, toute autre paire de vecteurs orthonormés appartenant à ce plan serait également un choix valide. Par exemple, nous pourrions effectuer une rotation des vecteurs u_2 et u_3 dans ce plan. Ceci produirait une matrice orthogonale différente Q' , mais l'égalité $A = Q' \Lambda Q'^\top$ resterait vérifiée.
- **Unicité du sous-espace :** Bien que l'on puisse modifier les vecteurs u_2 et u_3 , le sous-espace propre E_9 (le plan lui-même) est unique et invariant.

En conclusion, pour les matrices symétriques réelles, bien que la matrice Q ne soit pas unique en présence de valeurs propres multiples, la structure géométrique des sous-espaces propres est, quant à elle, parfaitement déterminée.

Décomposition En Valeurs Singulières

Soit $A \in \mathbb{R}^{m \times n}$ une matrice rectangulaire de rang r . Il existe une factorisation de la forme :

$$A = \underset{m \times n}{U} \cdot \underset{m \times m}{\Sigma} \cdot \underset{n \times n}{V}^\top$$

avec les propriétés suivantes :

- $U \in \mathbb{R}^{m \times m}$ est une matrice orthogonale. Ses colonnes u_i sont appelées les vecteurs singuliers à gauche.
- $V \in \mathbb{R}^{n \times n}$ est une matrice orthogonale. Ses colonnes v_j sont appelées les vecteurs singuliers à droite.
- $\Sigma \in \mathbb{R}^{m \times n}$ est une matrice rectangulaire dont les éléments diagonaux $\Sigma_{ii} = \sigma_i \geq 0$ sont appelés les valeurs singulières. Les valeurs singulières sont ordonnées par ordre décroissant : $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$.

De plus, la multiplication colonnes-lignes permet de séparer la matrice A en une somme de r matrices de rang 1 :

$$A = U\Sigma V^\top = \sigma_1 u_1 v_1^\top + \sigma_2 u_2 v_2^\top + \dots + \sigma_r u_r v_r^\top = \sum_{i=1}^r \sigma_i u_i v_i^\top$$

Chaque terme de cette somme, $\sigma_i u_i v_i^\top$, représente une matrice de rang 1. Dans une telle structure, toutes les colonnes de la matrice résultante sont des multiples scalaires du vecteur u_i . Par conséquent, la dimension de l'espace des colonnes est de 1, ce qui confirme que le rang de chaque matrice est exactement égal à 1.

Construction et Lien avec la Matrice Symétrique

Cette relation est fondamentale pour le calcul pratique de la DVS. Nous examinons la matrice symétrique $A^\top A$. Puisque $A^\top A$ est symétrique, la diagonalisation est possible grâce au Théorème Spectral. Supposons que $A = U\Sigma V^\top$. Calculons $A^\top A$:

$$\begin{aligned} A^\top A &= (U\Sigma V^\top)^\top (U\Sigma V^\top) \\ &= V\Sigma^\top \underbrace{U^\top U}_I \Sigma V^\top \\ &= V(\Sigma^\top \Sigma) V^\top. \end{aligned}$$

D'après le théorème spectral, la matrice symétrique $A^\top A$ est diagonalisable orthogonalement sous la forme $A^\top A = PDP^\top$. En identifiant les termes, nous obtenons deux résultats fondamentaux :

1. Les colonnes de V sont les vecteurs propres de $A^\top A$.
2. Les éléments diagonaux de $\Sigma^\top \Sigma$ qui est une matrice carrée (soit σ_i^2) sont les valeurs propres de $A^\top A$. Ainsi, $\sigma_i = \sqrt{\lambda_i}$.
3. Pour obtenir des vecteurs orthonormés u_i , il suffit de normaliser les vecteurs Av_i . La norme de Av_i est donnée par $\|Av_i\| = \sqrt{\lambda_i} = \sigma_i$. Ainsi, nous définissons les vecteurs singuliers à gauche comme suit :

$$u_i = \frac{Av_i}{\|Av_i\|} = \frac{1}{\sigma_i} Av_i.$$

Exemple 4. Considérons la matrice $A = \begin{bmatrix} 1 & 0 & 1 \\ -2 & 1 & 0 \end{bmatrix}$.

Calculons $A^\top A$ et ses vecteurs propres :

$$A^\top A = \begin{bmatrix} 5 & -2 & 1 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Les valeurs propres sont $\lambda_1 = 6, \lambda_2 = 1, \lambda_3 = 0$. Les vecteurs propres normalisés forment V :

$$v_1 = \frac{1}{\sqrt{30}}(5, -2, 1)^\top, \quad v_2 = \frac{1}{\sqrt{5}}(0, 1, 2)^\top, \quad v_3 = \frac{1}{\sqrt{6}}(-1, 2, 5)^\top.$$

Les valeurs singulières sont les racines carrées des valeurs propres non nulles :

$$\sigma_1 = \sqrt{6}, \quad \sigma_2 = 1. \implies \Sigma = \begin{bmatrix} \sqrt{6} & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}.$$

Nous utilisons la relation $u_i = \frac{1}{\sigma_i} A v_i$:

$$u_1 = \frac{1}{\sqrt{6}} A v_1 = \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \quad u_2 = \frac{1}{1} A v_2 = \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

On obtient ainsi la décomposition complète $A = U \Sigma V^\top$.

Enfin, illustrons la décomposition de A en une somme de matrices de rang 1 ($A = \sigma_1 u_1 v_1^\top + \sigma_2 u_2 v_2^\top$) :

$$\begin{aligned} A &= \sqrt{6} \left(\frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ -2 \end{bmatrix} \right) \left(\frac{1}{\sqrt{30}} \begin{bmatrix} 5 & -2 & 1 \end{bmatrix} \right) + 1 \left(\frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \right) \left(\frac{1}{\sqrt{5}} \begin{bmatrix} 0 & 1 & 2 \end{bmatrix} \right) \\ A &= \frac{\sqrt{6}}{\sqrt{150}} \begin{bmatrix} 5 & -2 & 1 \\ -10 & 4 & -2 \end{bmatrix} + \frac{1}{5} \begin{bmatrix} 0 & 2 & 4 \\ 0 & 1 & 2 \end{bmatrix} \end{aligned}$$

Sachant que $\frac{\sqrt{6}}{\sqrt{150}} = \frac{\sqrt{6}}{5\sqrt{6}} = \frac{1}{5}$, on obtient :

$$A = \begin{bmatrix} 1 & -0.4 & 0.2 \\ -2 & 0.8 & -0.4 \end{bmatrix} + \begin{bmatrix} 0 & 0.4 & 0.8 \\ 0 & 0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ -2 & 1 & 0 \end{bmatrix}$$

La première matrice de cette somme, $A_1 = \begin{bmatrix} 1 & -0.4 & 0.2 \\ -2 & 0.8 & -0.4 \end{bmatrix}$, représente la meilleure approximation de rang 1 de la matrice A .

[2]

Géométrie de la DVS

La DVS décompose une matrice sous la forme $A = U\Sigma V^T$: **(orthogonale)** $\times$ **(diagonale)** $\times$ **(orthogonale)**. Les matrices orthogonales U et V appliquent une rotation au plan. La matrice diagonale Σ l'étire le long des axes.

Afin d'illustrer visuellement les étapes de la DVS, nous avons généré la figure suivante à l'aide d'un script Python.

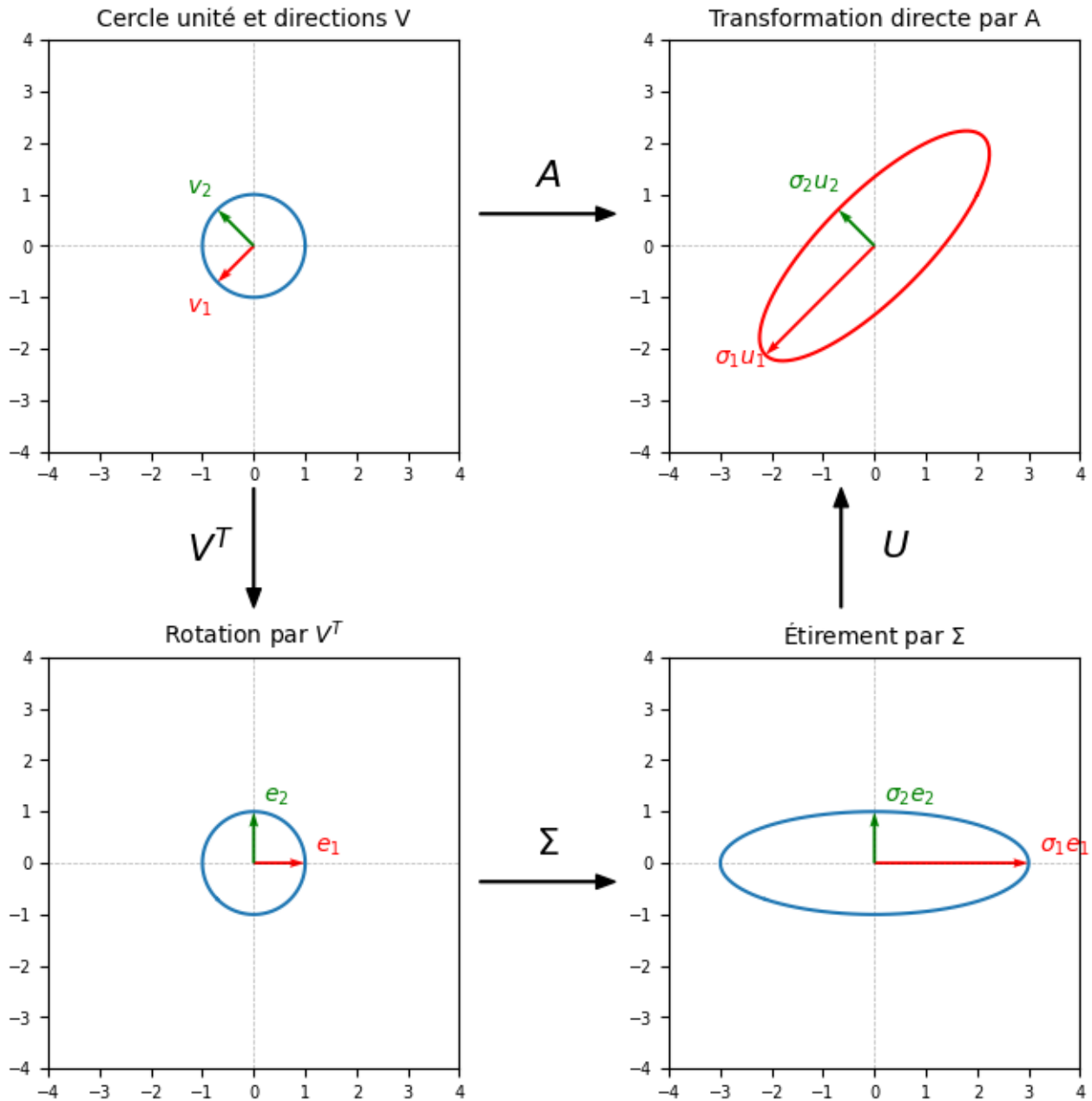


Figure 1: Illustration géométrique de la DVS générée via Python.

Le Premier Vecteur Singulier v_1

Nous avons défini les vecteurs singuliers à droite v_i comme étant les vecteurs propres de la matrice $A^\top A$. Nous comprendrons ces vecteurs singuliers un par un, de manière séquentielle, au lieu de les considérer tous en même temps. Commençons par le premier vecteur singulier v_1 et sa valeur singulière associée σ_1 .

Cette approche consiste à chercher la direction dans laquelle la matrice A étire le plus les vecteurs. Cela revient à maximiser le ratio suivant :

$$\max_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sigma_1 \quad (7)$$

Le Quotient de Rayleigh

Jusqu'à présent, nous avons défini la DVS en supposant l'existence de la décomposition. Supposons que nous ne connaissons ni U , ni Σ , ni V . Nous cherchons à maximiser le ratio $\frac{\|Ax\|}{\|x\|}$. Pour trouver un maximum, la méthode standard consiste à calculer la dérivée de la fonction et à l'égaliser à zéro. Comme la norme euclidienne contient une racine carrée, les calculs deviennent plus compliqués. Pour simplifier, on maximise le carré de la fonction (qui a le même maximum), ce qui nous conduit au Quotient de Rayleigh :

$$\max_{x \neq 0} \frac{\|Ax\|^2}{\|x\|^2} = \max_{x \neq 0} \frac{(Ax)^\top (Ax)}{x^\top x} = \max_{x \neq 0} \frac{x^\top A^\top A x}{x_1^2 + x_2^2 + \dots + x_n^2} = \max_{x \neq 0} \frac{x^\top S x}{x^\top x} \quad (8)$$

où $S = A^\top A$ est une matrice symétrique.

Pour trouver ce maximum, nous devons calculer les dérivées partielles par rapport à chaque composante x_i et les égaliser à zéro. Calculons d'abord les dérivées du dénominateur et du numérateur séparément :

$$\begin{aligned} \frac{\partial}{\partial x_i} (x^\top x) &= \frac{\partial}{\partial x_i} (x_1^2 + \dots + x_n^2) = 2x_i \\ \frac{\partial}{\partial x_i} (x^\top S x) &= \frac{\partial}{\partial x_i} \left(\sum_i \sum_j S_{ij} x_i x_j \right) = 2 \sum_j S_{ij} x_j = 2(Sx)_i \end{aligned}$$

Pour trouver ce maximum, nous devons calculer les dérivées partielles par rapport à chaque composante x_i et les égaliser à zéro. Calculons d'abord les dérivées du dénominateur et du numérateur séparément :

$$\begin{aligned} \frac{\partial}{\partial x_i} (x^\top x) &= \frac{\partial}{\partial x_i} (x_1^2 + \dots + x_n^2) = 2x_i \\ \frac{\partial}{\partial x_i} (x^\top S x) &= \frac{\partial}{\partial x_i} \left(\sum_i \sum_j S_{ij} x_i x_j \right) = 2 \sum_j S_{ij} x_j = 2(Sx)_i \end{aligned}$$

Ensuite, nous appliquons la règle de dérivation d'un quotient ($\frac{f'g - fg'}{g^2} = 0 \implies f'g - fg' = 0$). Pour que la dérivée globale soit nulle, le numérateur de cette dérivée doit obligatoirement s'annuler. Cela nous donne l'équation suivante :

$$x^\top x \cdot (2Sx) - (x^\top Sx) \cdot 2x = 0$$

Nous pouvons maintenant simplifier cette équation. En divisant chaque terme par 2 et en réorganisant, nous obtenons :

$$(x^\top x)Sx = (x^\top Sx)x$$

Divisons ensuite les deux côtés par $(x^\top x)$:

$$\frac{(x^\top x)Sx}{x^\top x} = \frac{(x^\top Sx)x}{x^\top x}$$

Ce qui se simplifie pour donner notre résultat final :

$$Sx = \left(\frac{x^\top Sx}{x^\top x} \right) x$$

Elle correspond exactement à la définition d'une équation aux valeurs propres ($Sx = \lambda x$). Par conséquent :

- Le vecteur x qui maximise notre ratio est par définition un vecteur propre de S (notre premier vecteur singulier v_1).
- Le terme entre grandes parenthèses, qui représente notre maximum, est la valeur propre λ correspondante (qui est égale à σ_1^2).

Exemple 5. Dans l'Exemple 8, nous avons effectué la décomposition en valeurs singulières de la matrice A , et nous avons trouvé le premier vecteur singulier v_1 ainsi que la valeur singulière maximale $\sigma_1 = \sqrt{6}$. Appliquons maintenant le Quotient de Rayleigh.

Rappelons notre matrice symétrique $S = A^\top A$ de l'exemple 8 :

$$S = \begin{bmatrix} 5 & -2 & 1 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Nous voulons montrer que le quotient de Rayleigh atteint sa valeur maximale dans la direction du premier vecteur singulier v_1 . Pour faciliter les calculs, il n'est pas nécessaire de rendre le vecteur unitaire ; en effet, dans le quotient de Rayleigh, la longueur du vecteur n'a pas d'importance, seule sa direction compte. C'est pourquoi nous utilisons directement notre vecteur non normalisé :

Étape 1 : Calcul du dénominateur (la norme au carré du vecteur) :

$$x^\top x = 5^2 + (-2)^2 + 1^2 = 25 + 4 + 1 = 30$$

Étape 2 : Calcul du produit matrice-vecteur Sx :

$$Sx = \begin{bmatrix} 5 & -2 & 1 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 5 \\ -2 \\ 1 \end{bmatrix} = \begin{bmatrix} (25 + 4 + 1) \\ (-10 - 2 + 0) \\ (5 + 0 + 1) \end{bmatrix} = \begin{bmatrix} 30 \\ -12 \\ 6 \end{bmatrix}$$

Étape 3 : Calcul du numérateur $x^\top Sx$:

$$x^\top Sx = \begin{bmatrix} 5 & -2 & 1 \end{bmatrix} \begin{bmatrix} 30 \\ -12 \\ 6 \end{bmatrix} = (5 \times 30) + (-2 \times -12) + (1 \times 6) = 150 + 24 + 6 = 180$$

Étape 4 : Quotient de Rayleigh :

$$\frac{x^\top Sx}{x^\top x} = \frac{180}{30} = 6$$

Le résultat de ce quotient est très exactement 6. Cela correspond à notre plus grande valeur propre $\lambda_1 = 6$, ce qui confirme que $\lambda_1 = \sigma_1^2 = (\sqrt{6})^2 = 6$. Nous avons ainsi vérifié numériquement que le quotient de Rayleigh trouve la variance maximale exactement dans la direction du premier vecteur singulier v_1 .

Définition 1 (Norme Spectrale d'une Matrice). Pour tout vecteur $x \in \mathbb{R}^n \setminus \{0\}$, la norme spectrale d'une matrice $A \in \mathbb{R}^{m \times n}$ est définie par :

$$\|A\|_2 := \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} \quad (9)$$

Définition 2 (Norme de Frobenius). Pour une matrice $A \in \mathbb{R}^{m \times n}$ de rang r , elle est définie par la racine carrée de la somme des carrés de ses valeurs singulières :

$$\|A\|_F = \sqrt{\sigma_1^2 + \sigma_2^2 + \cdots + \sigma_r^2} = \sqrt{\sum_{i=1}^r \sigma_i^2} \quad (10)$$

Théorème 6. La norme spectrale de A est sa plus grande valeur singulière σ_1 .

Démonstration. D'après la définition de la norme spectrale :

$$\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}.$$

Pour simplifier les calculs, nous en prenons le carré :

$$\|A\|_2^2 = \max_{x \neq 0} \frac{\|Ax\|_2^2}{\|x\|_2^2} = \max_{x \neq 0} \frac{x^\top A^\top A x}{x^\top x} = \max_{x \neq 0} R(x),$$

où $R(x)$ est le quotient de Rayleigh pour la matrice $S = A^\top A$.

Le maximum du quotient de Rayleigh est atteint dans la direction du vecteur propre correspondant à la plus grande valeur propre λ_1 de S . Par conséquent :

$$\|A\|_2^2 = \lambda_1.$$

D'après la DVS, les valeurs propres de S sont les carrés des valeurs singulières de A : $\lambda_1 = \sigma_1^2$. En conclusion :

$$\|A\|_2 = \sigma_1.$$

□

Approximation Matricielle

La Décomposition en Valeurs Singulières permet d'exprimer toute matrice A comme une somme de matrices de rang 1, chacune étant pondérée par sa propre valeur singulière (σ_i). L'ordre décroissant des valeurs singulières détermine directement le niveau de dominance de ces composantes sur la matrice.

Grâce à cette structure, il est possible d'approximer efficacement la matrice A en ne conservant qu'un nombre limité de termes (les k premiers) de cette somme. Cette approche nous conduit naturellement au concept fondamental d'approximation matricielle de rang faible (*low-rank approximation*), qui permet de préserver la grande majorité de l'information structurelle de la matrice d'origine.

Pour une matrice A de $\text{rang}(A) = r$, l'approximation de rang k (où $k < r$) est définie par l'expression suivante :

$$\hat{A}(k) = \sum_{i=1}^k \sigma_i u_i v_i^\top \quad (11)$$

Théorème 7 (Eckart–Young Théorème - Norme Spectral). *Soit $A \in \mathbf{R}^{m \times n}$ une matrice de rang r , et soit $B \in \mathbf{R}^{m \times n}$ une matrice de rang k , l'inégalité suivante est toujours satisfaite :*

$$\|A - B\|_2 \geq \|A - A_k\|_2 = \sigma_{k+1} \quad (12)$$

Ce théorème affirme que la matrice A_k , obtenue par la DVS, est la meilleure approximation de rang k de la matrice A au sens de la norme spectrale.

Démonstration. Supposons par l'absurde qu'il existe une matrice B de rang k telle que :

$$\|A - B\|_2 < \|A - A_k\|_2 = \sigma_{k+1} \quad (13)$$

Pour tout vecteur non nul $w \in \mathbf{R}^n$, nous aurions $\|(A - B)w\|_2 < \sigma_{k+1} \|w\|_2$. En particulier, pour tout $w \in \ker(B)$ (où $Bw = 0$), cela impliquerait :

$$\|Aw\|_2 < \sigma_{k+1} \|w\|_2 \quad (14)$$

D'autre part, considérons le sous-espace $V_{k+1} = \text{span}\{v_1, \dots, v_{k+1}\}$. Pour tout $w \in V_{k+1}$, w peut s'écrire comme $w = \sum_{i=1}^{k+1} c_i v_i$. Par l'orthogonalité des vecteurs v_i et puisque $\sigma_1 \geq \dots \geq \sigma_{k+1}$, nous avons :

$$\|Aw\|_2 = \left\| \sum_{i=1}^{k+1} c_i \sigma_i u_i \right\|_2 = \sqrt{\sum_{i=1}^{k+1} c_i^2 \sigma_i^2} \geq \sigma_{k+1} \sqrt{\sum_{i=1}^{k+1} c_i^2} = \sigma_{k+1} \|w\|_2 \quad (15)$$

D'après le théorème du rang, $\dim(\ker(B)) = n - k$ et nous savons que $\dim(V_{k+1}) = k + 1$. La somme de leurs dimensions étant $(n - k) + (k + 1) = n + 1 > n$, il existe nécessairement un vecteur $w \neq 0$ tel que $w \in \ker(B) \cap V_{k+1}$.

Cependant, pour ce vecteur w , les inégalités (14) et (15) sont contradictoires. Cette contradiction prouve qu'aucune matrice B de rang k ne peut surpasser l'approximation A_k . $\square$

Théorème 8 (Eckart–Young Théorème - Norme de Frobenius). *Soit $A \in \mathbf{R}^{m \times n}$ une matrice de rang r , et soit $B \in \mathbf{R}^{m \times n}$ une matrice de rang k , l'inégalité suivante est toujours satisfaite :*

$$\|A - B\|_F^2 \geq \|A - A_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

Ce théorème affirme que la matrice A_k , obtenue par la DVS, est la meilleure approximation de rang k de la matrice A au sens de la norme de Frobenius.

Démonstration. Toute matrice $B \in \mathbf{R}^{m \times n}$ de rang k peut s'écrire sous la forme d'une factorisation k $B = CR$, où $C \in \mathbf{R}^{m \times k}$ et $R \in \mathbf{R}^{k \times n}$.

On peut toujours paramétrer une telle factorisation en introduisant une décomposition DVS de B : $B = U_B \Sigma_B V_B^\top$, et en posant $C = U_B \Sigma_B$ et $R = V_B^\top$. Dans cette écriture, on a bien $RR^\top = I_k$ et les colonnes de C sont orthogonales, avec $C^\top C = \Sigma_B^2$ diagonale.

En utilisant l'identité $\|X\|_F^2 = \text{tr}(XX^\top)$, nous pouvons développer la fonction d'erreur $E = \|A - CR\|_F^2$ sous la forme suivante :

$$\begin{aligned} E &= \text{tr}((CR - A)(CR - A)^\top) \\ &= \text{tr}(CRR^\top C^\top) - \text{tr}(CRA^\top) - \text{tr}(AR^\top C^\top) + \text{tr}(AA^\top) \end{aligned} \quad (16)$$

Afin d'analyser la structure des solutions de rang k , on considère les conditions stationnaires par rapport à C et R .

Remarque 1. Pour le calcul des dérivées matricielles impliquant l'opérateur de trace (tr), nous utilisons les identités standards suivantes :

$$\begin{aligned} \frac{\partial}{\partial X} \text{tr}(XA) &= A^\top \\ \frac{\partial}{\partial X} \text{tr}(XAX^\top) &= XA^\top + XA \\ \frac{\partial}{\partial X} \text{tr}(AX^\top) &= A \end{aligned}$$

En dérivant par rapport à C , on obtient :

$$\frac{\partial E}{\partial C} = 2CRR^\top - 2AR^\top$$

De même, la dérivation par rapport à R donne :

$$\frac{\partial E}{\partial R} = 2C^\top CR - 2C^\top A$$

Aux points stationnaires, on a donc :

$$C = AR^\top, \quad C^\top A = (C^\top C)R$$

En combinant ces deux relations, on obtient :

$$(A^\top A)R^\top = R^\top (C^\top C)$$

Ce qui montre que les colonnes de $R^\top$ engendrent un sous-espace invariant de $A^\top A$, donc elles coïncident avec les vecteurs singuliers à droite associés aux k plus grandes valeurs singulières, i.e. $R^\top = V_k^\top$.

Réciproquement, on obtient :

$$(AA^\top)C = C(C^\top C)$$

Ce qui implique que les colonnes de C appartiennent au sous-espace engendré par les vecteurs singuliers à gauche U_k de A .

Ainsi, toute solution optimale de rang k est de la forme :

$$B = CR = U_k \Sigma_k V_k^\top = \sum_{i=1}^k \sigma_i u_i v_i^\top. \quad (17)$$

L'erreur associée s'écrit alors :

$$\|A - B\|_F^2 = \sum_{i \notin S} \sigma_i^2. \quad (18)$$

Pour minimiser cette quantité, il est nécessaire de choisir les indices correspondant aux plus grandes valeurs singulières de A . Ainsi, on prend

$$S = \{1, 2, \dots, k\}, \quad (19)$$

ce qui donne la meilleure approximation.

On obtient alors :

$$\|A - B\|_F^2 = \sum_{i=k+1}^r \sigma_i^2, \quad \text{et} \quad B = A_k = U_k \Sigma_k V_k^\top. \quad (20)$$

Cela conclut la démonstration du théorème d'Eckart-Young. $\square$

Application de la DVS à la Compression d'Images

Dans cette section, nous mettons en évidence le lien direct entre l'approximation matricielle optimale et la compression de données à l'aide de la décomposition en valeurs singulières (DVS). Une image en niveaux de gris peut être modélisée mathématiquement par une matrice réelle $A \in \mathbf{R}^{n \times m}$, où n et m représentent respectivement le nombre de pixels verticaux et horizontaux. Grâce à la DVS, la matrice originale est décomposée selon ses composantes singulières sous la forme $A = U \Sigma V^\top$.

D'après le théorème d'Eckart-Young, la meilleure approximation de rang k s'obtient en conservant uniquement les k plus grandes valeurs singulières et leurs vecteurs singuliers associés. Sous les normes spectrale et de Frobenius, les valeurs singulières négligées ($\sigma_{k+1}, \dots, \sigma_r$) sont alors interprétées mathématiquement comme du bruit (*noise*), ce qui permet de réaliser une réduction de dimension optimale tout en minimisant la perte d'information résiduelle.

Afin d'illustrer ce concept théorique en pratique, nous analysons deux images en niveaux de gris de dimensions 736×414 pixels, dont la matrice initiale est de rang égal à 414. Nous appliquons la DVS pour reconstruire les matrices approximatives A_k pour différentes valeurs de rang $k \in \{5, 10, 20, 50, 100\}$.

Enfin, nous procéderons à une analyse comparative des approximations obtenues pour ces deux images distinctes de mêmes dimensions, sous les mêmes valeurs de rang k .

chat simple.jpg - Approximations de rang k par DVS

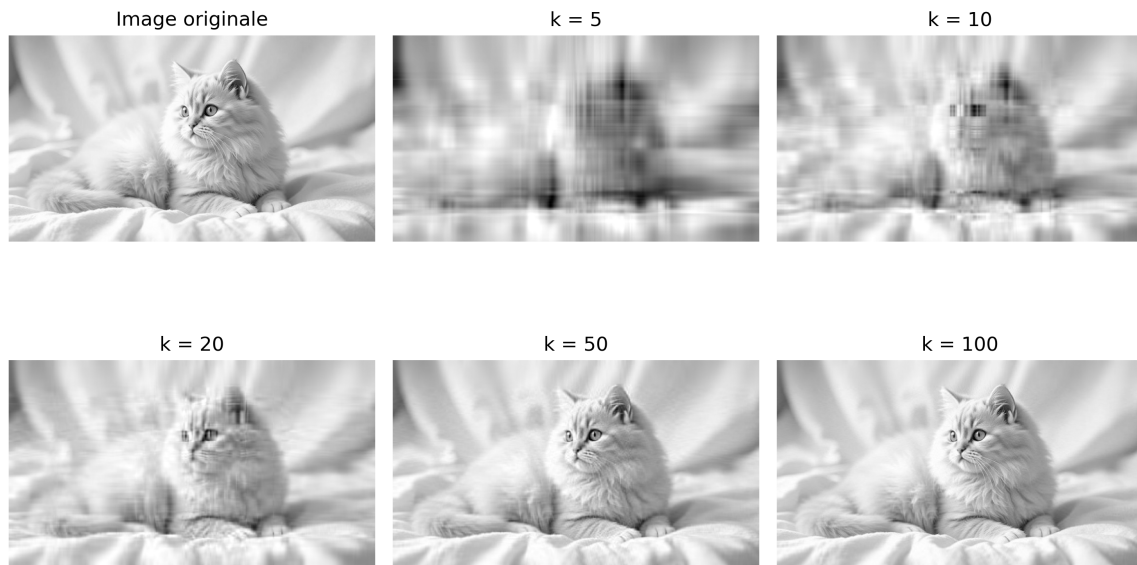


Figure 2

chat simple.jpg - Analyse DVS

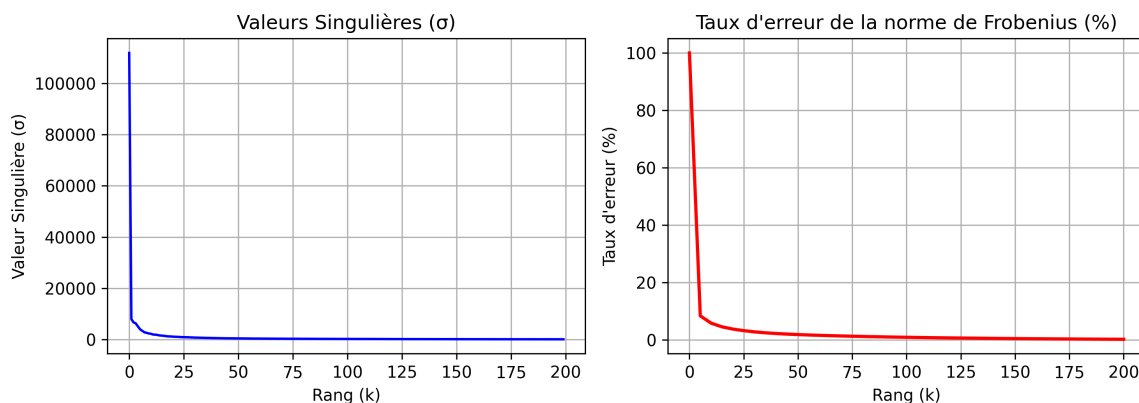


Figure 3

L'image montre un chat sur un arrière-plan doux (avec très peu de détails). En observant les différentes approximations de rang, l'évolution de la reconstruction est évidente :

- Pour $k = 5$ et $k = 10$, l'image est floue et présente d'importants effets de blocs.
- À $k = 20$, on distingue déjà très facilement la silhouette du chat.
- À $k = 50$, on obtient une image très proche de l'originale.
- À $k = 100$, la reconstruction est visuellement presque identique à l'image originale.

Sur le graphique des valeurs singulières, on observe une décroissance très rapide. Les toutes premières valeurs singulières sont largement dominantes, tandis que les suivantes diminuent drastiquement. Cela signifie que la plus grande partie de l'énergie (c'est-à-dire l'information essentielle de l'image) est concentrée dans ces premières valeurs singulières.

Le graphique de l'erreur relative suit exactement cette même tendance. Étant donné que les premières valeurs singulières contiennent l'essentiel de l'énergie de l'image et que les valeurs suivantes sont extrêmement petites, l'erreur tend très rapidement vers zéro à mesure que le rang k augmente.

chat détaillé.jpg - Approximations de rang k par DVS



Figure 4

chat détaillé.jpg - Analyse DVS

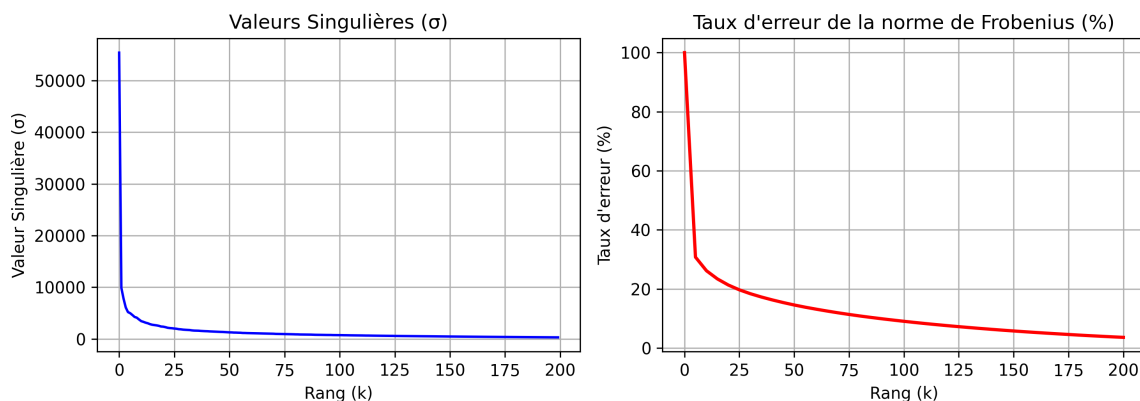


Figure 5

L'image montre un chat sur un arrière-plan complexe (végétation, texture dense). En observant les différentes approximations de rang, on constate que :

- À $k = 20$, on peut distinguer le chat et son environnement, mais la complexité des traits du visage du chat n'est pas encore résolue.
- À $k = 50$, le rendu est un peu plus net. Cependant, comparé au chat précédent (qui possédait un arrière-plan doux), la plupart des détails fins restent indiscernables.

Autrement dit, pour cette image complexe, une poignée de valeurs singulières ne suffit absolument pas à la représenter fidèlement.

Sur le graphique de l'erreur relative en fonction du rang, on retrouve une courbe décroissante similaire. Toutefois, par rapport au premier chat, l'erreur est nettement plus élevée, en particulier dans

l'intervalle de rang allant de 0 à 25. À titre d'exemple, pour la deuxième image, à $k \approx 25$, l'erreur est encore d'environ 20%, alors que pour la première image, au même rang k , elle est presque nulle.

Pour le premier chat, l'énergie est concentrée presque exclusivement dans les 20 premières valeurs singulières. Tandis que pour le deuxième, il faut atteindre $k \approx 50$ ou plus pour obtenir la même fidélité. L'énergie mathématique est donc répartie sur un bien plus grand nombre de valeurs singulières.

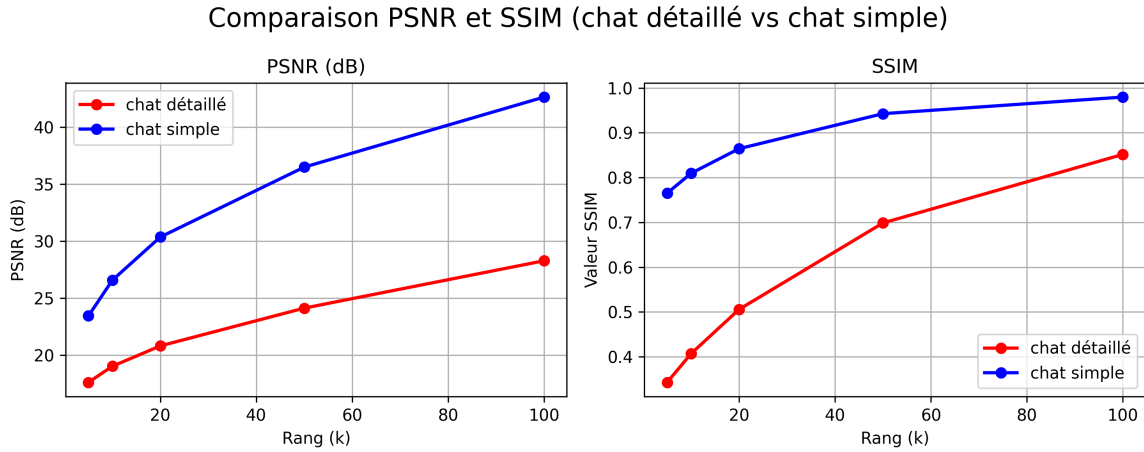


Figure 6

Le PSNR mesure à quel point l'image compressée ressemble à l'originale : plus il est élevé, meilleure est la qualité (son unité est le décibel, dB). Mathématiquement, pour une image en niveaux de gris où l'intensité maximale d'un pixel est de 255, le PSNR est défini à partir de l'erreur quadratique moyenne (MSE - *Mean Squared Error*) entre l'image originale I et l'image reconstruite K , toutes deux de dimensions $m \times n$:

$$\text{MSE} = \frac{1}{mn} \sum_{i=1}^m \sum_{j=1}^n [I(i, j) - K(i, j)]^2 \quad (21)$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) = 20 \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right) \quad (22)$$

- **> 40 dB** : Qualité presque sans perte (la différence n'est pas visible à l'œil nu).
- **Entre 30 et 40 dB** : Bonne qualité.
- **Entre 20 et 30 dB** : Qualité moyenne (la dégradation commence à se voir).
- **< 20 dB** : Faible qualité.

Sur le graphique, le PSNR augmente avec k pour les deux images, mais la courbe bleue (chat simple) est nettement au-dessus de la rouge (chat détaillé) en tout point. À $k = 20$, chat simple atteint environ 30 dB (bonne qualité) tandis que chat détaillé reste autour de 21 dB (moyenne). De plus, l'écart se creuse à mesure que k augmente : à $k = 100$, chat simple atteint environ 42 dB, soit une qualité presque parfaite, alors que chatdétaillé plafonne à environ 28 dB. Autrement dit, l'image simple atteint une haute qualité avec peu de composantes, tandis que l'image complexe n'atteint pas la même qualité même en ajoutant davantage de composantes.

Le SSIM mesure la ressemblance non pas pixel par pixel, mais à partir de la structure perçue par l'œil humain (forme générale, contraste). Il est donc plus proche de la perception visuelle humaine que le PSNR. Il prend une valeur comprise entre 0 et 1 :

- 1 : Identique à l'originale.
- $> 0,95$: Excellente qualité (presque indiscernable à l'œil).
- **Entre** 0,80 et 0,95 : Bonne qualité.
- $\approx 0,50$: Qualité moyenne (la dégradation est clairement visible).
- **Proche de** 0 : Structure très différente.

Sur le graphique, chat simple part déjà d'environ 0,77 à $k = 5$ (l'image est donc reconnaissable dès le départ) et atteint environ 0,94 à $k = 50$, où elle se stabilise presque. Au-delà de ce point, ajouter des composantes ne change plus guère la qualité. En revanche, chat détaillé reste très en retrait : à $k = 20$, elle n'atteint qu'environ 0,51 (à moitié ressemblante, l'image est encore floue), et même à $k = 100$, elle ne monte qu'à environ 0,85. L'écart le plus frappant s'observe à $k = 5$: chat simple est à environ 0,77 alors que chat détaillé est à environ 0,35.

Les deux donnent le même résultat : une image simple et peu bruitée atteint une haute qualité avec beaucoup moins de composantes, tandis qu'une image complexe et bruitée nécessite un nombre bien plus important de composantes pour atteindre cette même qualité, et parfois même, n'y parvient jamais complètement.

L'implication pratique est la suivante : si l'on fixe par exemple une contrainte de qualité de « minimum 30 dB pour le PSNR », un rang de $k \approx 20$ est amplement suffisant pour l'image simple. En revanche, pour l'image complexe, même à $k = 100$, ce seuil n'est pas atteint. Par conséquent, il n'existe pas un seul « bon » rang k universel ; le choix du rang k optimal dépend fondamentalement de la complexité intrinsèque de l'image à traiter.

Comparaison des Résultats

Dans l'intervalle de rang 50–100, on obtient pour les deux images des reconstructions très proches de l'originale, avec des taux d'énergie conservée très élevés : à $k = 50$, environ 99,9% pour chat simple et environ 98% pour chat détaillé — les deux dépassent donc largement 90%. Pourtant, ces pourcentages ne semblent pas réalistes au premier abord : en particulier pour chat détaillé (arrière-plan complexe), on voit clairement qu'à $k = 50$ beaucoup de petits détails manquent encore. Ce que dit le calcul (98% conservé) et ce que voit l'œil (image encore floue) semblent se contredire.

Cette contradiction vient de la manière dont l'« énergie » est définie. Dans une image en niveaux de gris, chaque pixel représente une intensité de luminosité comprise entre 0 et 255, et l'énergie totale est donnée par la norme de Frobenius, $\|A\|_F^2 = \sum_{i=1}^r \sigma_i^2$. Le taux d'énergie conservée par l'approximation de rang k est alors le rapport $\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$, c'est-à-dire les carrés des k valeurs singulières retenues divisés par ceux de toutes les valeurs singulières d'origine. Or le terme associé à la plus grande valeur singulière porte la structure grossière de luminosité de l'image (par exemple : « ici un arrière-plan clair, là une masse plus sombre qui est le chat ») : ayant une forte amplitude et s'étendant sur toute l'image, il porte à lui seul la plus grande part de l'énergie, mais ne contient aucun détail fin. Les détails fins — contours, texture du pelage, moustaches, densité du feuillage — sont codés dans les petites valeurs singulières de la queue de la distribution ; comme leurs carrés sont minuscules, leur contribution à l'énergie est négligeable, alors que c'est précisément ce que l'œil perçoit comme la netteté.

C'est pourquoi, à $k = 50$, dire « 98-99,9% de l'énergie est conservée » ne signifie pas « 98-99,9% de l'information visuelle est conservée ». La part restante de 0,1-2% n'est pas répartie au hasard : elle est concentrée précisément dans les détails fins, qui sont les plus importants pour la perception humaine.

Les graphiques PSNR et SSIM confirment cette analyse. Pour chat détaillé, « 98% d'énergie » semble parfait, mais une fois traduit en PSNR, cela ne donne à $k = 50$ qu'environ 24 dB (qualité moyenne) ; quant au SSIM, il ne reste qu'à environ 0,70. Ainsi, quand l'énergie dit « presque complet », le SSIM dit « structurellement encore incomplet de façon visible ».

Conclusion

Dans le cadre de ce travail, nous avons d'abord étudié les fondements du théorème spectral afin de comprendre la structure des matrices symétriques, ce qui nous a permis de généraliser ces concepts aux matrices rectangulaires en utilisant la décomposition en valeurs singulières (DVS). Grâce à l'analyse du quotient de Rayleigh et au théorème d'Eckart-Young, nous avons montré le rôle des valeurs propres et des valeurs singulières dans les problèmes d'approximation matricielle. Cette approche permet de comprendre comment une matrice peut être représentée de manière optimale par une approximation de rang faible, tout en minimisant la perte d'information selon les normes spectrale et de Frobenius.

Pour mettre cette théorie en pratique, nous l'avons appliquée à la compression d'images. Cet exemple met en évidence l'importance de la réduction de dimension, en montrant que les premières composantes singulières concentrent l'essentiel de l'information de l'image, tandis que les composantes restantes représentent des détails plus fins.

Dans notre application, les images ont été traitées en niveaux de gris. Car une image en niveaux de gris se représente par une seule matrice $A \in \mathbb{R}^{m \times n}$. Dans le cas d'une image en couleur (RGB), les données forment un tableau à trois dimensions ($m \times n \times 3$), qui n'est plus une matrice mais un tenseur. Comme la DVS n'est définie que pour les matrices bidimensionnelles, on l'applique alors séparément à chacun des trois canaux (rouge, vert et bleu), puis on recombine les résultats pour reconstruire l'image.

En résumé, cette méthode permet de simplifier et d'analyser des données complexes en conservant uniquement l'information la plus significative. Cela la rend particulièrement efficace dans de nombreux autres domaines tels que le traitement du signal, l'analyse de données, la vision par ordinateur et l'apprentissage automatique.

Annexes

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import matplotlib.patches as patches
4
5 A = np.array([[2, 1], [1, 2]])
6 U, S, Vt = np.linalg.svd(A)
7 Sigma = np.diag(S)
8
9 lim = 4
10 theta = np.linspace(0, 2*np.pi, 100)
11 circle = np.array([np.cos(theta), np.sin(theta)])
12
13 def setup_ax(ax, title):
14     ax.set_aspect('equal')
15     ax.set_xlim(-lim, lim)
16     ax.set_ylim(-lim, lim)
17     ax.axhline(0, color='black', lw=0.5, alpha=0.3, ls='--')
18     ax.axvline(0, color='black', lw=0.5, alpha=0.3, ls='--')
19     ax.set_title(title, fontsize=10)
20     ax.tick_params(labels=7)
21
22 fig, axs = plt.subplots(2, 2, figsize=(8, 8))
23 plt.subplots_adjust(wspace=0.5, hspace=0.5)
24
25 setup_ax(axs[0, 0], "Cercle unite et directions V")
26 axs[0, 0].plot(circle[0], circle[1], lw=1.5)
27 axs[0, 0].quiver(0, 0, Vt[0,0], Vt[0,1], color='red', angles='xy', scale_units='xy', scale=1)
28 axs[0, 0].quiver(0, 0, Vt[1,0], Vt[1,1], color='green', angles='xy', scale_units='xy', scale=1)
29 axs[0, 0].text(Vt[0,0]-0.6, Vt[0,1]-0.6, '$v_1$', color='red', weight='bold')
30 axs[0, 0].text(Vt[1,0]-0.6, Vt[1,1]+0.3, '$v_2$', color='green', weight='bold')
31
32 setup_ax(axs[0, 1], "Transformation directe par A")
33 trans_A = A @ circle
34 axs[0, 1].plot(trans_A[0], trans_A[1], color='red', lw=1.5)
35 v1f, v2f = A @ Vt[0], A @ Vt[1]
36 axs[0, 1].quiver(0, 0, v1f[0], v1f[1], color='red', angles='xy', scale_units='xy', scale=1)
37 axs[0, 1].quiver(0, 0, v2f[0], v2f[1], color='green', angles='xy', scale_units='xy', scale=1)
38 axs[0, 1].text(v1f[0]-1.0, v1f[1]-0.2, r'$\sigma_1 u_1$', color='red')
39 axs[0, 1].text(v2f[0]-1.0, v2f[1]+0.4, r'$\sigma_2 u_2$', color='green')
40
41 setup_ax(axs[1, 0], "Rotation par $V^T$")
42 trans_Vt = Vt @ circle
43 axs[1, 0].plot(trans_Vt[0], trans_Vt[1], lw=1.5)
44 axs[1, 0].quiver(0, 0, 1, 0, color='red', angles='xy', scale_units='xy', scale=1)
45 axs[1, 0].quiver(0, 0, 0, 1, color='green', angles='xy', scale_units='xy', scale=1)
46 axs[1, 0].text(1.2, 0.2, '$e_1$', color='red')
47 axs[1, 0].text(0.2, 1.2, '$e_2$', color='green')
48
49 setup_ax(axs[1, 1], "Etirement par $\Sigma$")
50 trans_Sigma = Sigma @ trans_Vt
51 axs[1, 1].plot(trans_Sigma[0], trans_Sigma[1], lw=1.5)
52 axs[1, 1].quiver(0, 0, S[0], 0, color='red', angles='xy', scale_units='xy', scale=1)
53 axs[1, 1].quiver(0, 0, 0, S[1], color='green', angles='xy', scale_units='xy', scale=1)
54 axs[1, 1].text(S[0]+0.2, 0.2, r'$\sigma_1 e_1$', color='red')
55 axs[1, 1].text(0.2, S[1]+0.2, r'$\sigma_2 e_2$', color='green')
56
57 plt.show()
```

Listing 1: Visualisation de la DVS avec Python

```

1 from PIL import Image
2 import numpy as np
3 img1 = Image.open("chat_simple.jpg").convert('L')
4 img2 = Image.open("chat_detaillée.jpg").convert('L')
5 print("Tailles originales :")
6 print(img1.size, img2.size)
7 target_size = (
8     min(img1.size[0], img2.size[0]),
9     min(img1.size[1], img2.size[1])
10 )
11 img1 = img1.resize(target_size, Image.LANCZOS)
12 img2 = img2.resize(target_size, Image.LANCZOS)
13 print("Taille commune :", target_size)
14 mat1 = np.array(img1, dtype=np.float64)
15 mat2 = np.array(img2, dtype=np.float64)
16 print("Shapes :", mat1.shape, mat2.shape)
17 rank1 = np.linalg.matrix_rank(mat1)
18 rank2 = np.linalg.matrix_rank(mat2)
19 print("Rank chat simple :", rank1)
20 print("Rank chat détaillée :", rank2)
21 print("Rank max théorique :", min(mat1.shape))

```

Listing 2: Chargement des images et calcul du rang matriciel

```

1 from PIL import Image
2 import numpy as np
3 image1_path = "chat_simple.jpg"
4 image2_path = "chat_detaillée.jpg"
5 img1 = Image.open(image1_path).convert('L')
6 img2 = Image.open(image2_path).convert('L')
7 target_size = (
8     min(img1.size[0], img2.size[0]),
9     min(img1.size[1], img2.size[1])
10 )
11 img1 = img1.resize(target_size, Image.LANCZOS)
12 img2 = img2.resize(target_size, Image.LANCZOS)
13 mat1 = np.array(img1, dtype=np.float64)
14 mat2 = np.array(img2, dtype=np.float64)
15 print("Shape :", mat1.shape)
16 def apply_svd_and_print(matrix, image_name):
17     U, S, Vt = np.linalg.svd(matrix, full_matrices=False)
18     total_energy = np.sum(S**2)
19     print(f"\nImage : {image_name}")
20     print(f"Singular values : {len(S)}")
21     print("\nTop 20 singular values")
22     for i in range(20):
23         print(f"sigma{i+1} = {S[i]:.2f}")
24     print("\nEnergy retention (%)")
25     for k in [5, 10, 20, 50, 100]:
26         energy = np.sum(S[:k]**2) / total_energy * 100
27         print(f"k={k:3d} -> {energy:6.2f}%")
28     return S
29 print("DVS analysis")
30 S1 = apply_svd_and_print(mat1, "chat_simple.jpg")
31 S2 = apply_svd_and_print(mat2, "chat_detaillée.jpg")

```

Listing 3: Calcul de l'énergie conservée via DVS

```

1 from PIL import Image
2 import numpy as np
3 image1_path = "chat_simple.jpg"
4 image2_path = "chat_detailed.jpg"
5 img1 = Image.open(image1_path).convert('L')
6 img2 = Image.open(image2_path).convert('L')
7 target_size = (
8     min(img1.size[0], img2.size[0]),
9     min(img1.size[1], img2.size[1])
10 )
11 img1 = img1.resize(target_size, Image.LANCZOS)
12 img2 = img2.resize(target_size, Image.LANCZOS)
13 mat1 = np.array(img1, dtype=np.float64)
14 mat2 = np.array(img2, dtype=np.float64)
15 print("Shape :", mat1.shape)
16 def analyze_svd(matrix, image_name):
17     U, S, Vt = np.linalg.svd(matrix, full_matrices=False)
18     total_energy = np.sum(S**2)
19     total_fro_norm = np.sqrt(total_energy)
20     print(f"\nImage : {image_name}")
21     print("k | Energy (%) | Frobenius error | PSNR (dB)")
22     print("-" * 70)
23     for k in [5, 10, 20, 50, 100]:
24         energy = np.sum(S[:k]**2) / total_energy * 100
25         fro_error = np.sqrt(np.sum(S[k:]**2)) / total_fro_norm
26         reconstructed = (
27             U[:, :k]
28             @ np.diag(S[:k])
29             @ Vt[:k, :]
30         )
31         mse = np.mean((matrix - reconstructed)**2)
32         if mse > 0:
33             psnr = 20 * np.log10(255 / np.sqrt(mse))
34         else:
35             psnr = float("inf")
36         print(
37             f"{k:3d} | "
38             f"{energy:10.2f}% | "
39             f"{fro_error:14.4f} | "
40             f"{psnr:9.2f}"
41         )
42     print("-" * 70)
43 print("DVS analysis start")
44 analyze_svd(mat1, "chat_simple.jpg")
45 analyze_svd(mat2, "chat_detailed.jpg")

```

Listing 4: Analyse de l'énergie, erreur de Frobenius et PSNR via DVS

```

1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4 image_path = "chat_simple.jpg"
5 original_img = Image.open(image_path).convert('L')
6 img = np.array(original_img, dtype=np.float64)
7 print("Matrix shape :", img.shape)
8 print("=" * 70)
9 print("APPROXIMATION DE RANG k (DVS)")
10 print("=" * 70)
11 print("A\approx A_k = U_k Sigma_k V_k^T")
12 print("U_k : k singular vectors (left)")
13 print("Sigma_k : diagonal matrix of k singular values")
14 print("V_k^T : k singular vectors (right)")
15 print("Eckart-Young theorem: best rank-k approximation")
16 print("=" * 70)
17 U, S, Vt = np.linalg.svd(img, full_matrices=False)
18 ks = [5, 10, 20, 50, 100]
19 fig, axes = plt.subplots(2, 3, figsize=(10, 7))
20 fig.suptitle("SVD rank-k approximations", fontsize=14)
21 axes[0, 0].imshow(original_img, cmap='gray')
22 axes[0, 0].set_title('Original')
23 axes[0, 0].axis('off')
24 for i, k in enumerate(ks):
25     approx = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
26     approx = np.clip(approx, 0, 255).astype(np.uint8)
27     row = (i + 1) // 3
28     col = (i + 1) % 3
29     axes[row, col].imshow(approx, cmap='gray')
30     axes[row, col].set_title(f'k = {k}')
31     axes[row, col].axis('off')
32 plt.tight_layout()
33 plt.savefig("chat_simple_svd_compression.png", dpi=300, bbox_inches='tight')
34 plt.show()
35 print("Saved: chat_simple_svd_compression.png")

```

Listing 5: Approximation de rang k et compression DVS

```

1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4 image_path = "chat_detailed.jpg"
5 original_img = Image.open(image_path).convert('L')
6 img = np.array(original_img, dtype=np.float64)
7 print("Matrix shape :", img.shape)
8 print("=" * 70)
9 print("RANK-k MATRIX APPROXIMATION (DVS)")
10 print("=" * 70)
11 print("A approx A_k = U_k Sigma_k V_k^T")
12 print("U_k : first k left singular vectors")
13 print("Sigma_k : diagonal matrix of k singular values")
14 print("V_k^T : first k right singular vectors")
15 print("Eckart-Young theorem: optimal rank-k approximation")
16 print("=" * 70)
17 U, S, Vt = np.linalg.svd(img, full_matrices=False)
18 ks = [5, 10, 20, 50, 100]
19 fig, axes = plt.subplots(2, 3, figsize=(12, 7))
20 fig.suptitle("SVD rank-k approximations (chat detaille)", fontsize=14)
21 axes[0, 0].imshow(original_img, cmap='gray')
22 axes[0, 0].set_title('Original image')
23 axes[0, 0].axis('off')
24 for i, k in enumerate(ks):
25     approx = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
26     approx = np.clip(approx, 0, 255).astype(np.uint8)
27     row = (i + 1) // 3
28     col = (i + 1) % 3
29     axes[row, col].imshow(approx, cmap='gray')
30     axes[row, col].set_title(f'k = {k}')
31     axes[row, col].axis('off')
32 plt.tight_layout()
33 plt.savefig("chat_detailed_svd_compression.png", dpi=300, bbox_inches='tight')
34 plt.show()
35 print("Saved: chat_detailed_svd_compression.png")

```

Listing 6: Approximation de rang k par DVS (image detaillee)

```

1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4 image_path = "chat_detaill .jpg"
5 img = Image.open(image_path).convert('L')
6 mat = np.array(img, dtype=np.float64)
7 print("Matrix shape :", mat.shape)
8 U, S, Vt = np.linalg.svd(mat, full_matrices=False)
9 ks = np.arange(0, 201, 5)
10 energy = np.zeros(len(ks))
11 fro_error = np.zeros(len(ks))
12 psnr_values = []
13 total_energy = np.sum(S**2)
14 for i, k in enumerate(ks):
15     if k == 0:
16         energy[i] = 0.0
17         fro_error[i] = 1.0
18         psnr_values.append(0.0)
19     else:
20         cum_energy = np.sum(S[:k]**2)
21         energy[i] = cum_energy / total_energy
22         fro_error[i] = np.sqrt(1 - energy[i])
23         recon = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
24         mse = np.mean((mat - recon)**2)
25         psnr = 20 * np.log10(255 / np.sqrt(mse)) if mse > 0 else 100
26         psnr_values.append(psnr)
27 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
28 fig.suptitle("SVD analysis - chat detaill ")
29 axes[0].plot(S[:200], linewidth=1.5)
30 axes[0].set_title("Singular values (sigma)")
31 axes[0].set_xlabel("k")
32 axes[0].set_ylabel("sigma")
33 axes[0].grid(True)
34 axes[1].plot(ks, fro_error * 100, linewidth=2)
35 axes[1].set_title("Frobenius error (%)")
36 axes[1].set_xlabel("k")
37 axes[1].set_ylabel("error (%)")
38 axes[1].grid(True)
39 plt.tight_layout()
40 plt.savefig("chat_detaill _svd_analysis_corrected.png", dpi=300, bbox_inches='tight')
41 plt.show()
42 print("Saved: chat_detaill _svd_analysis_corrected.png")

```

Listing 7: Analyse DVS : energie, erreur Frobenius et PSNR

```

1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4 image_path = "chat_simple.jpg"
5 img = Image.open(image_path).convert('L')
6 mat = np.array(img, dtype=np.float64)
7 print("Matrix shape :", mat.shape)
8 U, S, Vt = np.linalg.svd(mat, full_matrices=False)
9 ks = np.arange(0, 201, 5)
10 energy = np.zeros(len(ks))
11 fro_error = np.zeros(len(ks))
12 psnr_values = []
13 total_energy = np.sum(S**2)
14 for i, k in enumerate(ks):
15     if k == 0:
16         energy[i] = 0.0
17         fro_error[i] = 1.0
18         psnr_values.append(0.0)
19     else:
20         cum_energy = np.sum(S[:k]**2)
21         energy[i] = cum_energy / total_energy
22         fro_error[i] = np.sqrt(1 - energy[i])
23         recon = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
24         mse = np.mean((mat - recon)**2)
25         psnr = 20 * np.log10(255 / np.sqrt(mse)) if mse > 0 else 100
26         psnr_values.append(psnr)
27 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
28 fig.suptitle("SVD analysis - chat simple")
29 axes[0].plot(S[:200], linewidth=1.5)
30 axes[0].set_title("Singular values (sigma)")
31 axes[0].set_xlabel("k")
32 axes[0].set_ylabel("sigma")
33 axes[0].grid(True)
34 axes[1].plot(ks, fro_error * 100, linewidth=2)
35 axes[1].set_title("Frobenius error (%)")
36 axes[1].set_xlabel("k")
37 axes[1].set_ylabel("error (%)")
38 axes[1].grid(True)
39 plt.tight_layout()
40 plt.savefig("chat_simple_svd_analysis_corrected.png", dpi=300, bbox_inches='tight')
41 plt.show()
42 print("Saved: chat_simple_svd_analysis_corrected.png")

```

Listing 8: Analyse DVS : image simple

```

1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from skimage.metrics import structural_similarity as ssim
5 print("=" * 80)
6 print("PSNR AND SSIM DEFINITIONS")
7 print("=" * 80)
8 print("PSNR: pixel-based error metric")
9 print("PSNR = 20 * log10(255 / sqrt(MSE))")
10 print("Higher PSNR = better reconstruction\n")
11 print("SSIM: structural similarity metric")
12 print("Measures luminance, contrast, structure")
13 print("Range: 0 to 1 (1 = identical image)")
14 print("=" * 80)
15 def compute_metrics(image_path, name):
16     original = Image.open(image_path).convert('L')
17     orig_array = np.array(original, dtype=np.float64)
18     U, S, Vt = np.linalg.svd(orig_array, full_matrices=False)
19     ks = [5, 10, 20, 50, 100]
20     psnr_list = []
21     ssim_list = []
22     orig_uint8 = np.clip(orig_array, 0, 255).astype(np.uint8)
23     print(f"\nResults for {name}")
24     print("k | PSNR | SSIM")
25     print("-" * 40)
26     for k in ks:
27         recon = U[:, :k] @ np.diag(S[:k]) @ Vt[:, k, :]
28         recon = np.clip(recon, 0, 255).astype(np.uint8)
29         mse = np.mean((orig_array - recon)**2)
30         psnr = 20 * np.log10(255 / np.sqrt(mse)) if mse > 0 else 100
31         ssim_val = ssim(orig_uint8, recon, data_range=255)
32         psnr_list.append(psnr)
33         ssim_list.append(ssim_val)
34         print(f"{k:3d} | {psnr:8.2f} | {ssim_val:.4f}")
35     return ks, psnr_list, ssim_list
36 ks, psnr_detail, ssim_detail = compute_metrics("chat_detaill e.jpg", "detaill e")
37 ks, psnr_simple, ssim_simple = compute_metrics("chat_simple.jpg", "simple")
38 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
39 fig.suptitle("PSNR and SSIM comparison")
40 axes[0].plot(ks, psnr_detail, 'r-o', label='detaill e')
41 axes[0].plot(ks, psnr_simple, 'b-o', label='simple')
42 axes[0].set_title('PSNR')
43 axes[0].set_xlabel('k')
44 axes[0].set_ylabel('PSNR')
45 axes[0].legend()
46 axes[0].grid(True)
47 axes[1].plot(ks, ssim_detail, 'r-o', label='detaill e')
48 axes[1].plot(ks, ssim_simple, 'b-o', label='simple')
49 axes[1].set_title('SSIM')
50 axes[1].set_xlabel('k')
51 axes[1].set_ylabel('SSIM')
52 axes[1].legend()
53 axes[1].grid(True)
54 plt.tight_layout()
55 plt.savefig("psnr_ssim_comparison.png", dpi=300, bbox_inches='tight')
56 plt.show()
57 print("Saved: psnr_ssim_comparison.png")

```

Listing 9: Comparaison PSNR et SSIM via DVS

Références

- [1] Sheldon Axler. *Linear algebra done right*. Springer Nature, 2024.
- [2] Marc Peter Deisenroth, A Aldo Faisal, and Cheng Soon Ong. *Mathematics for machine learning*. Cambridge University Press, 2020.
- [3] Department of Mathematics, University of Texas at Austin. Singular value decomposition. <https://web.ma.utexas.edu/users/sl55444/Archive/SVD.pdf>. Consulté le 19/12/2025.
- [4] S. Friedli. Algèbre linéaire. <https://botafogo.saitis.net/pdfs/algebre-lineaire.pdf>, 2024. Polycopié de cours, EPFL. Consulté le 29/10/2025.
- [5] Ian T Jolliffe. Principal component analysis: a beginner's guide—i. introduction and application. *Weather*, 45(10):375–382, 1990.
- [6] Jun Lu. Matrix decomposition and applications. *arXiv preprint arXiv:2201.00145*, 2022.
- [7] M Pelini and V Ledda. Matrice d'une application linéaire. 2018.
- [8] UNISCIEL. Valeurs propres - vecteurs propres - diagonalisation d'une matrice carrée. https://uel.unisciel.fr/physique/outils_nancy/outils_nancy_ch11/co/apprendre_ch11_20.html, 2023. Université des Sciences en Ligne. Consulté le 14/01/2026.